

บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในโรงงาน

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆ ที่เกี่ยวข้องในการทำโรงงาน ซึ่งเนื้อหาในบทนี้จะกล่าวถึงเทคนิคและทฤษฎีขั้นพื้นฐาน ซึ่งเนื้อหาทั้งหมดนี้จำเป็นสำหรับการศึกษา เพื่อนำไปใช้ในการพัฒนาโรงงาน

2.1 Color Model

Color model ใช้สำหรับอ้างอิงถึงสีต่างๆ โดยไม่ใช่ analytical model เหมือนกับที่ใช้ทางวิทยาศาสตร์ซึ่งใช้วิธีการวัดให้อยู่ในรูปของพลังงานตลอดช่วงของสเปกตรัมแต่จะเป็น empirical model ที่ได้รับความสัมพันธ์ของค่าที่ใช้อ้างอิงกับสีใดๆ จากการทดลองที่ศึกษาแบบ psychophysical ที่มีการรับรู้ของมนุษย์เข้ามาเกี่ยวข้องสื่อสารนั้น โดยมีรายละเอียด model แบบต่างๆ ดังนี้

2.1.1 RGB Model

RGB Model เป็น color model ที่เฉพาะเจาะจงกับอุปกรณ์ใดอุปกรณ์หนึ่ง ซึ่งในที่นี้ก็คือจอภาพตัวใดตัวหนึ่งของคอมพิวเตอร์เท่านั้น เนื่องจาก RGB model ได้ทำการสร้างสีต่างๆ ขึ้นโดยใช้แหล่งกำเนิดแสงจำนวนสามสีได้แก่ Red Green Blue ที่เกิดจากสารเรืองแสงที่มีคุณสมบัติที่แตกต่างกันตามลำดับ ซึ่ง Red Green และ Blue นี้ก็จะไม่เท่ากันในแต่ละอุปกรณ์นอกเสียจากว่าจะมีคุณสมบัติของสารเรืองแสงและการ setup จอและสภาพแวดล้อมที่จอภาพตั้งอยู่ที่เหมือนกันทุกประการ ซึ่งโดยปกติแล้วก็จะมีความแตกต่างกันออกไป

การที่เราเห็นภาพจากจอภาพเหล่านั้นได้ก็เพราะการทำงานของคลื่นแม่เหล็กไฟฟ้าเรียงตัวกันตามลำดับ (แดง-เขียว-น้ำเงิน) หรือที่เรียกว่า Spectrum โดยอัตราของสีที่แตกต่างกัน และเมื่อทับซ้อนกัน (Colors Overlap) ก็จะเกิดเป็นสีอื่นๆ อีกมากมาย เมื่อทำการแสดงสีที่มีค่าเท่ากับ $[1.0, 0.0, 0.0]$ ซึ่งเป็นสีแดงจะหมายถึงสีแดงของจอภาพนั้นๆ และเมื่อนำไปเปรียบเทียบกับ $[1.0, 0.0, 0.0]$ จอภาพอื่นๆ ก็จะได้สีแดงที่มีค่าไม่เท่ากับจอภาพสีแดงตัวแรก ดังนั้นเมื่อทำการเปรียบเทียบสี RGB ของจอภาพมากกว่าสองตัวขึ้นไปจะต้องเปรียบเทียบโดยใช้ XYZ Model ที่ไม่ขึ้นกับอุปกรณ์ โดยการแปลง RGB ของจอภาพตัวที่หนึ่งไปเป็น XYZ จากนั้นจึงทำการแปลง RGB ของจอภาพตัวที่สองอีกครั้งที่ต้องทำเช่นนี้เนื่องจากไม่สามารถที่จะทำการเปรียบเทียบระหว่างจอภาพทั้งสองได้โดยตรง เนื่องจากไม่มีข้อมูลความสัมพันธ์ของสีระหว่างจอภาพทั้งสองแต่เรามีความสัมพันธ์ของข้อมูลสีของจอภาพแต่ละตัวกับระบบสี XYZ ที่เป็นมาตรฐานซึ่งค่าดังกล่าวสามารถทราบได้จากผู้ผลิตจอภาพนั้น

ดังนั้นระบบสี RGB จึงเป็นระบบสีที่ขึ้นอยู่กับอุปกรณ์แสดงผล(device dependent) อีกทั้งยังไม่สามารถที่จะแสดงสีบนสเปกตรัมได้ทั้งหมดแต่ก็นับว่ามี color gamut ที่เพียงพอในการคำนวณในงานคอมพิวเตอร์กราฟิกสามมิติ เนื่องจากอยู่ในรูปของ intensity ซึ่งสมการในการคำนวณของ illumination model จะคำนวณในรูปของ intensity แต่การกำหนด color model แบบ RGB นี้ก็ถือว่าเป็นเหมาะสมกับทุกกรณีเสมอไปจึงได้มีการคิดค้น color model อื่นที่ช่วยให้มีความเหมาะสมกับงานอื่นๆมากยิ่งขึ้น

ระบบสี RGB ประกอบด้วยข้อมูลจำนวนสามส่วนคือค่า intensity ของสีใน 3 channel ได้แก่ Red Green และ Blue ตามลำดับ โดยสีดังกล่าวก็จะแตกต่างกันไปในแต่ละจอภาพดังที่กล่าวไว้ข้างต้น การที่เราจะสร้างสีหรือ reproduce สีต่างๆ โดยใช้สี RGB ดังกล่าวนี้นั้นสามารถที่จะกระทำได้โดยการแสดงสีทั้งสามสีที่มี intensity ที่แตกต่างกันออกไปทำให้เรามองเห็นเป็นสีต่างๆ เนื่องจาก RGB ที่มี intensity ที่แตกต่างกันนี้จะไปกระตุ้น cone cell แต่ละชนิดมากน้อยต่างกันไปและทำให้มีการรับรู้ว่าเป็นสีต่างๆ โดยกระบวนการประสาทต่อไป

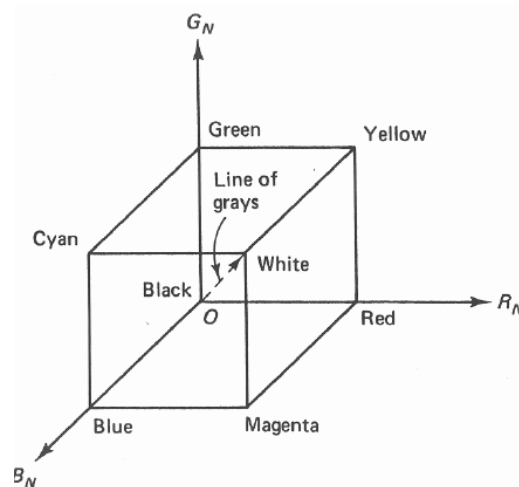
สัญลักษณ์ที่ใช้แทน RGB ในงานคอมพิวเตอร์กราฟิกนั้นสามารถที่จะแสดงได้โดยการใช้วงเล็บเพื่อแสดงค่าเช่น (R,G,B) หรือ [R,G,B] เป็นต้น ค่าที่แสดงในวงเล็บดังกล่าวสามารถเป็นได้ทั้ง floating point และ fixed point ในกระบวนการ render นั้น สี RGB จะถูกกำหนดชนิดของข้อมูลในการคำนวณด้วยคอมพิวเตอร์ให้เป็นแบบ floating point หรือเลขทศนิยม เนื่องจากต้องการความละเอียดสูง เช่น อาจถูกกำหนดเป็น [0.23564f,0.56546f,0.98465f] เป็นต้นโดยค่า intensity ในแต่ละ channel จะอยู่ระหว่าง 0 ถึง 1 แต่เนื่องจากการกำหนดแหล่งกำเนิดแสงนั้น intensity ของแหล่งกำเนิดแสงสามารถที่จะมีค่าได้มากกว่า 1 ดังนั้นภายหลังจากการคำนวณค่า intensity อาจจะมีค่ามากกว่า 1 ได้ในแต่ละ channel ซึ่งค่าใน channel ใดๆที่มีค่าเกิน 1 ก็จะถูกตัด (clamp) ให้มีค่าไม่เกิน 1 แหล่งกำเนิดแสงสามารถที่จะถูกกำหนดให้เป็นค่าลบได้เช่นเดียวกันและทำนองเดียวกันค่า intensity ที่น้อยกว่า 1 ก็จะถูกตัดให้มีค่าที่ไม่ต่ำกว่าศูนย์เช่นกัน โดยงานที่มีการจัดการเรื่องแสงเป็นอย่างดีจะไม่นิยมให้เกิดการ clamp ในลักษณะนี้เพราะอาจจะทำให้ภาพมีการไล่ tone ของแสงไม่ถูกต้องสวยงาม และไม่สามารถที่จะแก้ไขได้ในภายหลังเนื่องจากไม่สามารถทราบข้อมูลสีที่โดนตัดไป

การเก็บข้อมูลสี RGB ให้อยู่ในรูป floating point นี้มีข้อดีเมื่อเราต้องการที่จะดำเนินการใดๆบนภาพที่เราสร้างขึ้น เนื่องจากมี precision ที่สูงทำให้ข้อมูลสีมีการบิดเบือนที่น้อยแต่ต้องใช้เนื้อที่ในการเก็บที่มาก เช่น 1 pixel ถ้าจัดเก็บในรูป RGBA ด้วยข้อมูลชนิด float ต้องใช้เนื้อที่เท่ากับ $4 \times 4 = 16$ bytes/pixel หรือ 32bits/channel ซึ่งปริมาณที่มากเกินไปจนความจำเป็นถ้าเราต้องการที่จะเก็บภาพเพื่อทำการแสดงผลเพียงอย่างเดียว เนื่องจากจอภาพทั่วไปนั้นสามารถแสดงสีที่แตกต่างกันได้เพียง 10 ถึง 12 bit เท่านั้น อีกทั้งตาของมนุษย์สามารถที่จะแยกแยะสีต่างๆได้ทั้งหมดประมาณ 10 ล้านสีและจอภาพก็ไม่สามารถที่จะแสดงสีเหล่านั้นได้ทั้งหมด ดังนั้นการใช้ข้อมูลสีที่มีขนาดเท่ากับ 8bit/channel หรือ 256 ระดับในแต่ละ channel ที่สามารถแสดงสีที่

แตกต่างกันได้ทั้งหมด 16.7 ล้านสีนั้น ก็เพียงพอที่จะแสดงสีที่มนุษย์เห็นได้อย่างเป็นธรรมชาติ และใช้เนื้อที่ในการเก็บข้อมูลสีเพียง 4bytes/pixel เท่านั้น

ด้วยเหตุนี้การใช้ระบบ RGB ทั่วไปจึงกำหนดค่า intensity เป็น 8 bits ซึ่งเท่ากับ 256 ระดับในแต่ละ channel เช่น $[0,0,0]$ หมายถึง intensity มีค่าเท่ากับ 0 ในทุก channel หรือสีดำและ $[255,255,255]$ ซึ่งเท่ากับสีขาว ดังนั้นเมื่อพูดถึงค่าที่เป็นข้อมูลสีในแต่ละ channel ของ color model ใดๆในบางครั้งจึงอาจจะใช้คำว่าสูงสุดหรือต่ำสุดแทน เนื่องจากค่าสูงสุดอาจเป็นได้ทั้ง 1.0 เมื่อมีช่วงข้อมูลเท่ากับ 0 ถึง 1 หรืออาจจะเป็น 255 เมื่อมีช่วงข้อมูลเท่ากับ 0 ถึง 255 หรืออาจจะเป็น 100% เมื่อมีช่วงข้อมูล 0 ถึง 100%

เพื่อช่วยให้เกิดความเข้าใจใน RGB model และสามารถมองเห็นภาพได้ดียิ่งขึ้นจึงได้มีการสร้าง RGB cube ขึ้นโดยเป็น cube ที่ถูกสร้างในพิกัด 3 มิติ โดยให้แกนทั้งสามเป็นค่าของ intensity ในแต่ละ channel ตามลำดับ ดังนั้นจึงทำให้เกิด cube ที่มีมุมทั้งหมด 8 มุมที่เป็นตำแหน่งของ Red Green Blue White และ Cyan Magenta Yellow Black จำนวน 8 สีตามลำดับ โดยชุดแรก RGB เป็น primary color ของการผสมสีแบบ additive และชุดหลัง CYM คือ yprimary color ของการผสมสีแบบ subtractive

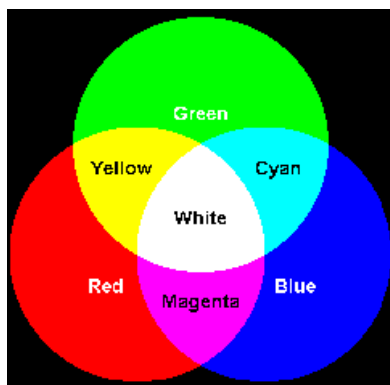


รูปที่ 2.1 RGB cube

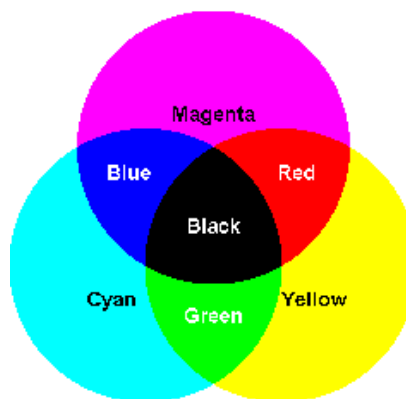
ใน RGB cube นี้มีเส้นที่ลากจุด $[0,0,0]$ ไปยัง $[255,255,255]$ เรียกว่า ซึ่งเป็นเส้นที่ลากจากสีดำไปจนถึงสีขาว ซึ่งสีที่อยู่บนเส้นนี้จะเป็นสีเทาที่มี intensity ที่แตกต่างกันออกไปโดยค่า intensity ของสีบน line of gray ดังกล่าวจะมีค่า intensity ที่เท่ากันทั้งสาม channel ดังแสดงในรูปที่ 2.1

การสร้างสีทำได้ 2 วิธีคือ การผสมสีแบบ additive ได้แก่ การผสมสีโดยใช้แสงสีโดยเริ่มจากสีดำซึ่งก็คือความมืดหรือไม่มีแสง และทำการเติมหรือเพิ่มแสงสีทั้งสามสีได้แก่ RGB ลงไปเพื่อให้ได้สีที่ต้องการไปจนกระทั่งถึงสีขาว ดังแสดงในรูปที่ 2.2 ส่วนการผสมสีแบบ subtractive

ซึ่งการผสมสีนั้นจะเริ่มต้นจากสีขาวโดยใช้แสงสีขาวส่องลงบนกระดาษก่อนจากนั้นจึงทำการกำจัดหรือลบสีที่ไม่ต้องการออกจากแสงสีขาวยุ่่นโดยการใช้สารสี CYM เพื่อทำการดูดกลืนสีที่ไม่ต้องการออกไปทำให้เกิดเป็นสีต่างๆและไปสิ้นสุดที่สีดำ ดังที่แสดงในรูปที่ 2.3



รูปที่ 2.2 Additive Color



รูปที่ 2.3 Subtractive color

2.1.2 HSL Model

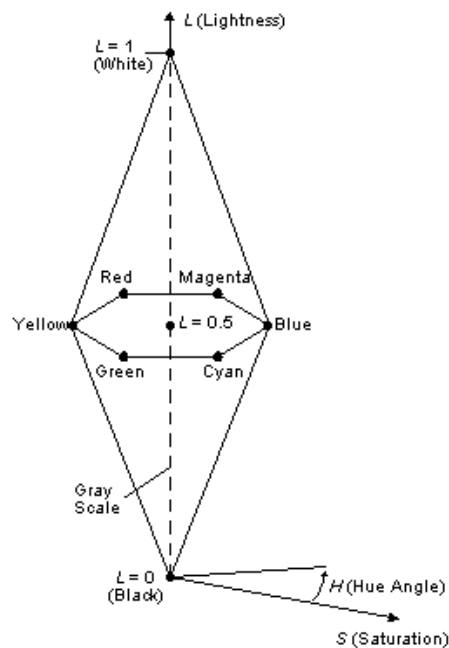
HSL color model เป็น color model ที่นิยมใช้กันมากโดยมีคุณสมบัติหลายอย่างเหมือนกับ HSV โดยคำนึงถึงการใช้ในงานศิลปะเช่นเดียวกัน ระบบสี HSL นี้ประกอบด้วยสามส่วนคือ

H หมายถึง hue หรือสีที่มีค่าแตกต่างกันออกไปตามความถี่ของแสงเช่น แดง น้ำเงิน ม่วง เขียว เป็นต้น

S หมายถึง saturation หรือ ความอิ่มตัวของ hue นั้นๆ เช่น สีแดง และสีชมพูก็คือสีแดง เพียงแต่สีชมพูมีความอิ่มตัวที่น้อยกว่าหรือมีสีขาวเข้ามาผสม

L หมายถึง lightness หรือ ค่าความสว่างของสีที่มีค่า hue และ saturation เท่าใดโดยที่ lightness ต่ำสุดหมายถึงสีดำ ไม่ว่าจะเป็น hue และ saturation เท่าใด แล lightness สูงสุดหมายถึงสีขาว ไม่ว่าจะเป็น hue และ saturation เท่าใด

เมื่อนำ HSL model มาสร้างเป็น model ในพิกัด 3 มิติจะทำให้ได้รูปร่างที่มีลักษณะของ hexagonal cone สองอันประกบกันโดยใช้ด้านที่เป็นฐานในการประกบและมีจุดยอดอยู่ที่ปลายด้านบน และด้านล่างตามลำดับ ปลายด้านบนที่มีค่า lightness เท่ากับ 1 ก็คือ สีขาวและปลายด้านล่างที่มีค่า lightness เท่ากับ 0 ก็คือสีดำ สีที่อิ่มตัวที่สุดจะอยู่บนระนาบที่ lightness มีค่าเท่ากับ 0.5 โดยอยู่ตรงกลางระหว่างปลายด้านทั้งสองด้าน นอกจากนั้นแล้วยังมี gray scale นั่นคือ แกนกลางของ cone ทั้งสองอัน รูปร่าง double hexagonal cone ดังกล่าวคือ รูปร่างที่ transform มาจาก RGB cube ของ RGB model ซึ่งมีมุมเท่ากับ 8 มุม แต่ละมุมจะมีสีที่เหมือนกับ RGB cube ได้แก่ RGBW และ CYMK ดังแสดงในรูปที่ 2.4



รูปที่ 2.4 HSL Color Model

2.2 Chroma keying

เทคนิคการนำภาพ 2 ภาพมารวมกันซึ่งทำให้เกิดเป็นภาพใหม่ขึ้นมา โดยบางส่วนได้มาจากภาพแรกส่วนที่เหลือก็มาจากภาพที่สอง. A mask ในที่นี้คือส่วนหนึ่งของภาพที่ 1 มีหลายวิธีที่ทำให้ a mask ยังคงอยู่ วิธีหนึ่งที่ใช้บ่อยในการผลิตสื่อทางโทรทัศน์นั่นก็คือ Chroma Keying

ภาพ 1 ภาพที่ทำโดยวิธีของ chroma keying ก็จะถูกบันทึกใน studio ซึ่งจะมี background ที่เป็นสีเดียวกันโดยทั่วไปแล้วจะใช้สีน้ำเงิน นักแสดงจะต้องสวมเสื้อผ้าที่มีสีแตกต่างจากฉากพื้นหลังสำหรับรายการโทรทัศน์จะใช้อุปกรณ์พิเศษในการทำให้เกิด a mask ที่สอดคล้องกับส่วนของภาพที่เป็นสีน้ำเงินซึ่งสามารถใช้ในการรวมภาพของนักแสดงกับ background ที่มีความแตกต่างกันโดยใช้คอมพิวเตอร์เช่น รายการพยากรณ์อากาศซึ่งจะมีภาพแผนที่ภูมิแสดงสภาพลมฟ้าอากาศมาแทนที่ background ซึ่งแผนที่นั้นถูกสร้างโดยคอมพิวเตอร์

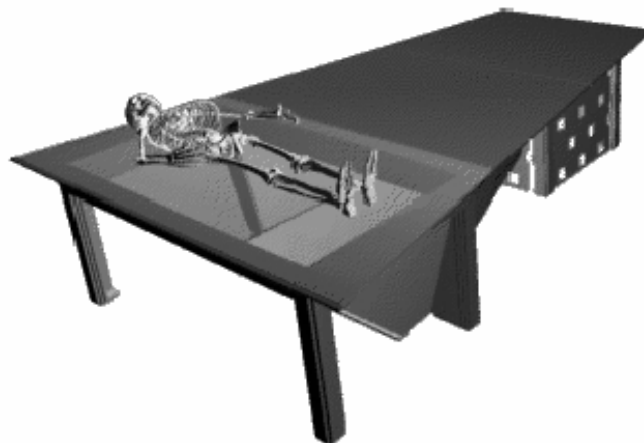
ในบางที่จะใช้เทคนิคของ chroma key ในรูปแบบของ software เพื่อขจัดปัญหาในเรื่องของ hardware ชนิดพิเศษที่มีราคาแพงขั้นตอนที่น่าเสนอนี้เป็นวิธีการที่ค่อนข้างง่าย,รวดเร็วและคุณภาพดีเมื่อเทียบกับภาพที่ได้จากการทำด้วย hardware

ในหัวข้อที่ 2.2.1 จะกล่าวถึงลักษณะโดยทั่วไปของ Immersive Telepresence และ chroma keying และในหัวข้อที่ 2.2.2 เป็นการบรรยายถึง algorithm ของ chroma keying ส่วนผลลัพธ์ที่ได้จะกล่าวถึงในหัวข้อที่ 2.2.3 และหัวข้อที่ 2.2.4 จะกล่าวถึงประโยชน์ของ software chroma keying

2.2.1 Virtual Environment

Projective Display System (PDS)เป็นลักษณะของศิลปะขั้นสูงของ Virtual Environment ระบบ PDS จะมีการทำงานที่หนักและส่งผลกระทบต่อ head-mounted display, ต้องมีความแน่นอนและต้องมีความรวดเร็วในการ rendering ซึ่งเหมาะที่จะใช้ hardware ลักษณะพิเศษของ projection-base VE system คือ ส่วนทั้งหมดของ real space ต่อ virtual space ทำงานร่วมกันกับ world co-ordinate system. Projection-based VR

การปะติดปะต่อ virtual space ไปยัง real space ต้องใช้ projection-base virtual environments เช่น โต๊ะ หรือห้องที่ถูกสร้างขึ้น โดยทั่วไปโต๊ะหรือห้องที่ถูกสร้างขึ้น ที่เป็นมาตรฐานยอมรับโดยทั่วกันจะต้องผ่านการจดลิขสิทธิ์ Responsive Workbench(RWB คือ การจดทะเบียนการค้าของ German Nation Research center for Information Technology), Collaborative Responsive Workbench the CyberStage หรือ Teleport. RWB ถูกพัฒนาขึ้นเมื่อปี 1993 ที่ GMD และใช้โต๊ะในการทำงานซึ่งเหมาะที่จะแสดงเกี่ยวกับภาพจำลอง 3 มิติ และการที่ตอบสนองดูเป็นธรรมชาติมากกว่า แสดงในรูปที่ 2.5



รูปที่ 2.5 A model of the Responsive Workbench

และใน Collaborative Responsive Workbench นั้นได้เพิ่มเติมในส่วนของ Responsive Workbench โดยเพิ่มจากแนวตั้งขึ้น 1 อัน ดังนั้นการขยายภาพจะมีมุมมองในแบบ frustum(ส่วนที่เหลื่อของรูปกรวยที่ถูกตัดออกโดยขนานกับพื้น) ดังแสดงในรูปที่ 2.6 แต่ละภาพที่แสดงมีขนาด 1.80m x 1.20m และสังเกตว่า frustum อยู่รอบๆ 2m ต่อ 3m ใน cube นี้การจำลองวัตถุ 3 มิติ สามารถถูกจับต้องได้โดยตรงโดยผ่านทางเข็ม,ทางข้อมูลของถุงมือหรืออุปกรณ์ input อื่นๆ Collaborative Responsive Workbench นอกจากจะมีมุมมองที่ใหญ่กว่าแล้ว ยังสามารถทำงานร่วมกันได้ในระยะไกลที่มากกว่า กล่าวคือการทำงานรอบโต๊ะนั่นเอง



รูปที่ 2.6 Collaborative Responsive Workbench

CyberStage คือ CAVE (การจดทะเบียนการค้าของ University of Illinois) มักจะมีระบบ room-size stereo display 4 ด้าน ติดตั้งที่ GMD ผู้ใช้จะเห็น virtual space ขนาดใหญ่และได้ยินเสียงที่กระจายออกมา Projection system นั้นคล้ายกับ CyberStage โดยสามารถเปลี่ยนแปลงได้โดยตรงละคนที่ศูนย์กลางจะติดต่อกับ virtual world ได้เป็นอย่างดี ขนาดของฉาก 3 ฉากที่ถูกติดตั้งขึ้นกับพื้นแต่ละอันมีขนาด 3x3 เมตร Crystal Eyes เป็นเลนส์ที่ใช้สำหรับ stereo ภาพที่แสดงมีขนาด 1024x768 pixel ที่ 10 Hz ระบบเสียง 8 ทิศทาง โดย IRCMP's room ซึ่งใช้ software Spatilisateur ในการบันทึกเสียง และรองรับเสียงต้นฉบับที่จำกัดเขตภายใน Virtual Environment วิธีการใหม่ๆ ของ CyberStage คือ acoustic floor ซึ่งรองรับการสร้างฉากที่สัมผัสเสมือน

วิธีการแรกในการ Immersive Telepresence คือการ mask โดยใช้ Mask ของ Collaborative Responsive Workbench และ CyberStage ในการจัดทำ virtual space. Immersive Telepresence อาศัยหลักการทำงานของ stereo-video, hardware chromakey และองค์ประกอบการ mapping ในการสั่งให้ video รวมกับ virtual environment

Chroma-keying คือเทคนิคที่ใช้ร่วมกับ virtual studio ผู้แสดงจะอยู่ในพื้นที่ของห้องที่มีสีน้ำเงิน chroma-keyer ใช้แยกภาพนักแสดงออกมาจาก background สีน้ำเงิน ทำโดยเพิ่ม alpha-channel ไปใน frame video ต้นฉบับ. นำ video มารวมกับ virtual environment ในเวลา real-time. Camera parameter คือ track และ virtual environment คือการ render ในเวลา real-time ซึ่งขึ้นอยู่กับเทคนิคการเขียนภาพของ real camera ถึงแม้ว่าอุปกรณ์ของ Chroma keying จะมีคุณภาพสูงสำหรับ virtual studio แต่มันยังมีราคาแพง จึงมีการเสนอวิธีการใช้ software chroma keying

2.2.2 Chroma Keying Exact Solutions

การเข้าถึงปัญหาเกี่ยวกับคณิตศาสตร์ของ chroma keying ถูกเสนอโดย Smith และ Blinn พวกเขาให้คำจำกัดความและเรียกมันว่า “Matting Problem” ซึ่งต่อจากนั้นเขาได้อธิบายถึงการใช่วิธีการใหม่

ให้ C_f และ C_b อยู่จุด corresponding และ C_k คือสีพื้นหลัง สมมุติว่า $C_f = C_o + (1 - \alpha_0) C_k$ และให้ C_o เป็นส่วนประกอบของสี $C = C_o + (1 - \alpha_0) C_b$ ที่จุด corresponding, สำหรับทุกจุดนั้น C_f และ C_b ใช้ร่วมกัน Note C_i คือ vectors $C_i = [R_i \ G_i \ B_i \ \alpha_i]$ ค่าของ α_i อยู่ในช่วง $[0 \dots \alpha_i]$ โดยที่ $0 \leq \alpha_i \leq 1$

C_i คือสีวัตถุที่แสดงใน foreground ซึ่งรวมด้วยส่วนประกอบ alpha. นี่คือน้ำหนักที่กำหนดให้สี backing C_k และสีที่แท้จริงของภาพที่จุดนั้นๆ, C_f จะยอมให้วัตถุรวมกับ background ที่กำหนด(แทนโดย C_b) การแก้ปัญหามatting problem เริ่มต้นโดยตัวประกอบภาพ(ซึ่งถูกบันทึกโดยกล้อง)และรู้ถึงสี backing (เช่นสีน้ำเงิน) ผลของระบบคือ

$$R_f = R_o + (1 - \alpha_0) R_k$$

$$G_f = G_o + (1 - \alpha_0) G_k$$

$$B_f = B_o + (1 - \alpha_0) B_k$$

X_f เป็นตัวประกอบที่แทนด้วยค่าที่ถูกบันทึกโดยกล้อง และ X_k แทน background X_o จะมาจากการแก้ปัญหของระบบนี้ Note บันทึกว่ามีค่าที่ไม่รู้, และเพียงสามสมการเท่านั้นที่ส่งผลภายใต้การเจาะจงระบบดังนั้นมันเป็นไปได้ที่จะรับการแก้ปัญหในเรื่องปัญหา chroma keying เมื่อการใช้การเข้าถึงนี้โดยการบังคับการจัดวางบนระบบกลายเป็นการแก้ปัญหของมัน เช่น ต้องการวัตถุ foreground ที่ไม่ใช่สีน้ำเงินเป็นส่วนประกอบ ผลคือ

$$C_o = \begin{bmatrix} R_f & G_f & 0 & 1 - \frac{B_f}{B_k} \end{bmatrix}$$

โดยมีตัวเลือกในการจำกัดเป็นคณิตศาสตร์คือ

$$C_o = \begin{bmatrix} R_f & G_f & B_f - B_k + \alpha_0 B_k & \frac{G_f - (B_f - B_k)}{B_k} \end{bmatrix}$$

Smith และ Blinn แสดงให้ดูเพียงอย่างเดียว การแก้ปัญหาล้มเหลวไปคงอยู่ แต่วิธีของพวกเขาต้องการถ่ายวัตถุที่แตกต่างกับ background ซึ่งวิธีการนี้ไม่สามารถใช้ได้เมื่อมีนักแสดงเข้ามาาร่วมด้วย

2.2.3 Chroma Keying: Approximation

สิ่งสำคัญในการทำงานบน chroma keying คือมันมีการป้องกันโดยการจดสิทธิบัตรส่วน มามันจะหมดอายุภายในเวลาไม่กี่ปีสิ่งที่กล่าวนี้ถูกค้นพบโดย Petro Vlahos ผู้ซึ่งรู้ดีเกี่ยวกับ Ultimatte™ ของเขา. วิธีการประมาณค่าต้องใช้นุญ้ยในการแก้ปัญหาด้านคณิตศาสตร์(หรืออาจจะ ใช้ software intelligent) ในการปรับค่าตัวแปรบางตัวจนกระทั่งได้ผลลัพธ์ที่ถูกต้อง ส่วนหนึ่งที่ ยากที่สุดในการประมาณค่าคือ

$$\alpha_o = 1 - a_1(B_f - a_2 G_f),$$

สำหรับการประมาณค่าของการทำงานนี้มีข้อสมมุติฐานหลายอย่างที่ต้องทำ เช่น การใช้ background สีน้ำเงิน,ผู้แสดงจะต้องไม่สวมเสื้อผ้าสีน้ำเงิน เป็นต้น ค่าที่ใกล้เคียง chroma key ลำดับที่สองอยู่ที่ความพอใจมากกว่า สมมุติฐานแรกก็คือ ช่วงของ background color c_k นั้น จะอยู่ในพื้นที่ขอบเขตของ 3D-RGB และเป็นส่วนที่ขาดหายไปจากสีที่พบในส่วน foreground color object รอบๆ ขอบเขตในพื้นที่ RGB จะมีรูปหลายเหลี่ยมมากมายถูกสร้างขึ้นมาซ้อนทับกัน อันที่เล็กกว่าอันนี้จะประกอบกันเป็นสีของ background ที่จู่รวมภายในจะถูกกำหนดให้มีค่าอัลฟา ของศูนย์ รูปหลายเหลี่ยมที่ใหญ่ที่สุดต้องอยู่บนหรืออยู่นอกเหนือพื้นที่ของสีที่พบใน foreground object นั่นคืออัลฟาเท่ากับหนึ่งสำหรับที่ทุกจุดที่อยู่นอกเหนือรูปหลายอันนี้ ในส่วนที่อยู่ระหว่าง รูปหลายเหลี่ยมสองอันจะเกิดสิ่งที่แตกต่างกันขึ้น แต่ละอันจะถูกแทนที่ด้วยค่าคงที่อัลฟาของ พื้นที่ผิวหน้าอยู่ระหว่าง 0-1

อุปกรณ์ primatte™ ของบริษัท Photron Ltd จะขึ้นอยู่กับค่าพวกนี้โดยใช้ 128-faced polyhedra นี่เป็นส่วน of ความรู้เบื้องต้นที่พูดถึงการทำงานของระบบ Smith และ Bill ได้พูดถึง ปัญหาที่ยังคงเหลืออยู่สำหรับการประมาณค่าการเข้าใกล้นี้

สำหรับ algorithm ที่มีอยู่ในปัจจุบันนั้นค่อนข้างมาได้ถูกเก็บไว้ในลำดับของการคาดคะเน ความต้องการของเราจะอยู่ระหว่างการปรับตั้งค่า จนกว่าจะได้ผลลัพธ์ที่พอใจ

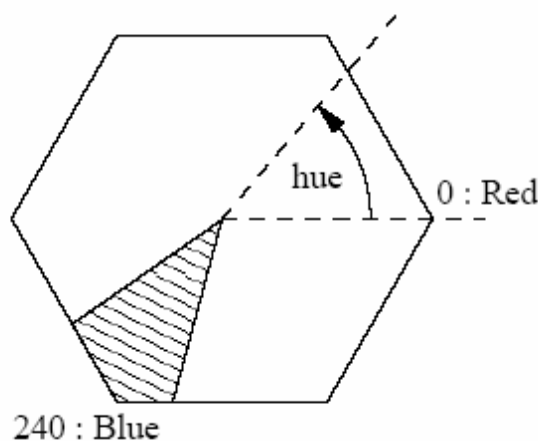
2.2.4 Algorithm

สิ่งที่สำคัญที่สุดของขั้นตอน chroma keying algorithm คือการแยก pixel ว่าเป็น key color หรือไม่ สำหรับการเปรียบเทียบ hue base ที่เกิดขึ้นนั้นมีการกล่าวถึงตั้งแต่แรกตามหัวข้อ การปรับปรุง ส่วนที่แตกต่างกันระหว่าง 2 algorithm นี้ขึ้นอยู่กับ การแบ่งแยกสีที่ทำสำเร็จ

สำหรับเรื่องนี้ key color นี้ถูกสมมุติให้เป็นสีน้ำเงินว่าเป็นสีที่นิยมใช้เป็น key color ซึ่ง ใกล้เคียงกับสีผิวมนุษย์ จะพบได้ว่าผู้คนทั้งหลายชอบที่จะทำงานในห้องสีน้ำเงินซึ่งต่างจากสีแดง กับสีเขียว

2.2.5 Hue-base approaches

คอมพิวเตอร์ส่วนใหญ่จะแสดงผลโดยใช้ RGB นี่คือนี่ที่สามารถใช้แสดงแทนรูปได้ ที่การเปลี่ยนแปลงง่าย ๆ ของ RGB สามารถที่จะแปลงไปจนใช้แทนที่ HSL ซึ่งประกอบไปด้วย สี, แสง, และ มีน้ำมีนวล ส่วนที่เป็นรูปทรงเลขาคณิตของ พื้นที่สี HSL นั้นคือ double-hexcone ถ้า มอง hexcone จากด้านบนแล้ว เราจะเห็น hexcone ที่มีสีเปลี่ยนแปลงไปตามมุมของเส้น วัดจากสีแดงที่มุม 0 องศา ในส่วนที่จัดไว้ สีน้ำเงิน จะอยู่ที่ 240 องศา ด้วยเหตุนี้ การกำหนดจุดสีน้ำเงินเราสามารถที่จะกำหนดคร่าวๆระหว่าง 230-255 องศา ดังในรูปที่ 2.7 เช่น รูป 6 เหลี่ยม HSL ส่วนที่เป็นเงานั้นจะถูกระบาย ส่วนที่เป็นสีนั้นจะเปลี่ยนไปตามมุม และควรที่จะถูกเลือกโดย ผู้ใช้ นำมาเทียบกับรายละเอียดของ ส่วนที่เป็นฉากสีน้ำเงิน



รูปที่ 2.7 HSL hexagon

ในระหว่างที่ใกล้จะเรียบร้อยดีนั้น จะเกิดอุปสรรคเมื่อใช้ software รูปภาพนั้นจะอยู่ในส่วนของ HSL ถ้าไม่จะต้องแปลงมันก่อนโดยใช้ slow down อัลกอริทึม

สำหรับการเปรียบเทียบการใช้ software HSL นั้นได้มีอยู่แล้ว อัลกอริทึมจะมีประโยชน์มาเมื่อใช้สีน้ำเงินเท่านั้น นั่นหมายความว่า HSL เต็มขั้นนั้นห้ามทำ หัวใจของ อัลกอริทึมนั้นดูได้จาก รูปที่ 2.8 จำไว้ว่าการ implementation มีแค่ สอง ส่วนเท่านั้น คือ ส่วนที่โป่งแสงและส่วนที่ทึบแสง

```

// s points to the interleaved RGBA image
unsigned char *rp=s,*gp=s+1,*bp=s+2,*ap=s+3;
int h,l,s;
for (int idx = 0; idx < imgsize; idx++)
{
    int max = (*rp > *bp) ? *rp : *bp;
    max = max > *gp ? max : *gp;
    int min = (*rp < *bp) ? *rp : *bp;
    min = min < *gp ? min : *gp;

    if (max == min) h = UNDEFINED; else
    {
        int delta = max - min;
        if (*bp == max) h = 240 +
            (60*(*rp-*gp))/delta;
        else h = 0;
        if (h < 0) h += 360;
    }

    if ((h < hue_max) && (h > hue_min))
        *ap = 0;
    else *ap = 255;
    rp += 4; bp += 4;
    gp += 4; ap += 4;
}

```

รูปที่ 2.8 C++ code for the HSL algorithm

2.2.6 An improved approach

ถ้าเรามีภาพที่อยู่ในพื้นที่ RGB มันเป็นไปได้ที่จะเพิ่มความเร็วของอัลกอริทึมสูงสุดถึง 5 operation ต่อ pixel และต้องพิจารณาให้ได้ว่าให้ได้ว่า pixel ใดนั้นมีสีฟ้า อย่างแรกที่เราต้อง set ให้เป็นมาตรฐานคือ $B > R$ และ $B > G$ ซึ่ง RGB เป็นส่วนประกอบของภาพที่สามารถแยกออกมาได้ง่ายในระยะ [0...255] กล่าวอีกอย่างหนึ่ง, ถ้าจุดภาพ pixel จะดูเหมือนสีน้ำเงิน, ส่วนประกอบสีที่เป็นจุดเด่นต้องเป็นน้ำเงิน อย่างไรก็ตาม มันยังคงกว้างเกินไป เพราะเราสามารถมี $B = x$, $R = x-1$, $G = x-1$ ซึ่งเป็นเฉดของสีเทาจะถูกแยกคิดเป็นสีน้ำเงินแทน มีทางหนึ่งโดยจำกัดการตั้งค่าให้แคบลงโดยเพิ่มการบังคับระยะ เช่น ถ้า

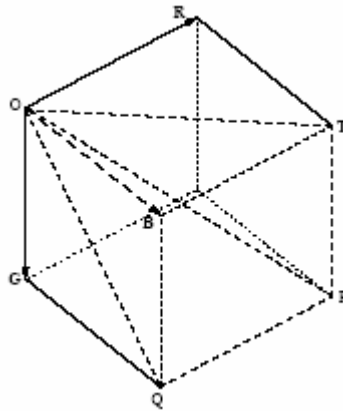
$$d = \sqrt{(B-R)^2 + (B-G)^2} > d_{max} \quad (1)$$

ต่อมาเราสามารถพูดถึงนั้นจุดภาพพิกลจะปรากฏสีน้ำเงินอย่างเห็นได้ชัด, และควรจะมี key ออก การคูณและการถอดสแควร์รูทจะเสียเวลามาก, ดังนั้นทำให้ง่ายขึ้นโดยใช้สมการที่ (2) มาแทน

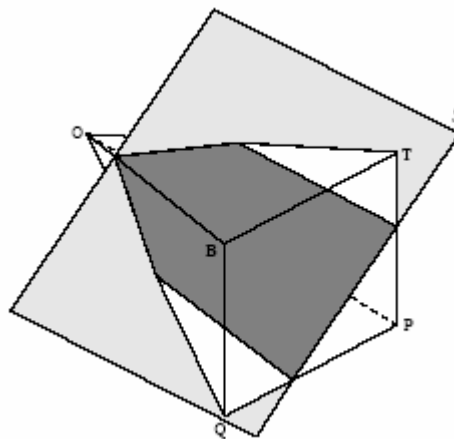
$$d = 2 \times B - R - G \quad (2)$$

ในรูปที่ 2.9 นี้จะแสดง RGB cube ซึ่งภายในมีพีระมิดที่ลาดเอียง OBTPQ (เส้นที่อยู่ข้างใน) พีระมิดนี้จะจำกัดค่าของ cube ซึ่งคือ $B \geq R$ และ $B \geq G$ ในรูปที่ 2.10 จะแสดง

พีระมิด OBTPQ ที่มีระนาบ S ตัดผ่านโดยระนาบ S นี้จะขนานกับเส้นทแยงมุมหลัก OP และแสดงพื้นผิวของค่าคงที่ d (ดังที่กำหนดไว้ในสมการที่ (2)) Note สมการนี้ถูกจำกัดเพียงแค่ภาพในพีระมิด(ส่วนของสีน้ำเงินเท่านั้น)



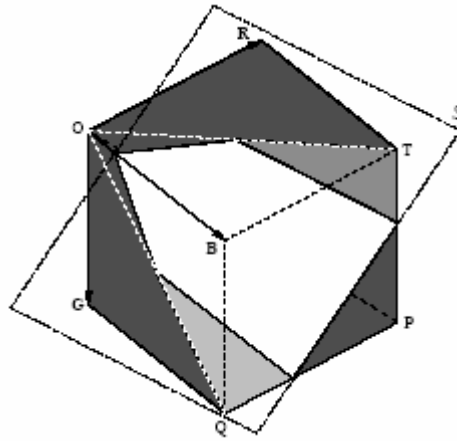
รูปที่ 2.9 An RGB cube showing the skew pyramid OBTPQ



รูปที่ 2.10 An RGB cube showing a plane S intersecting the skew pyramid OBTPQ

Dark-gray polygon คือ พื้นผิวหน้าตัดของพีระมิดและระนาบ S จุดทั้งหมดภายใน polygon นี้มีลักษณะเช่นเดียวกับสีในพื้นที่ RGB space กับ ค่าคงที่ d (ค่า d ที่กำหนดในสมการที่(2)) เส้นทแยงมุม OP คือ “gray line” ของ cube , สีทั้งหมดบนเส้นนี้มีค่าเท่ากับ R,G และส่วนประกอบ B ระนาบ S จะขนานกับเส้นทแยงมุม OP ,ความจริงที่ได้มาจากสมการที่(2) สิ่งนี้ชี้แนะให้เห็นว่าจุดทั้งหมดใน dark-gray polygon มีค่าเท่ากับสีน้ำเงิน การประมาณค่าเท่ากับค่าความเข้มของค่า gray และเท่ากับค่าความเข้ม ดังนั้นค่า d ของสีในพีระมิดสามารถแยกลำดับชั้นเริ่มต้นจากสีน้ำเงินหรือไม่ก็เลือกโดยจุดเริ่มต้นในค่า d Note ถึงแม้ค่า d จะมีระยะไม่ถูกต้องตามเมตริก, แต่ค่อนข้างจะคำนวณการประมาณมีประสิทธิภาพทำให้มองเห็นระดับที่เหนือกว่าระนาบ S ก่อนที่จะเคลื่อนไปยังรูปที่ 2.11

ในรูปที่ 2.11 จะแสดงถึง RGB cube อีกครั้ง ใน cube นี้พีระมิดยังคงมองเห็นเส้นทแยงมุม ทำให้มองเห็นรูปภาพที่ระดับของพีระมิดเหนือกว่าระนาบ S ที่ถูกย้าย สามเหลี่ยมสองด้านที่มีสีเทาเป็นด้านข้างของกำแพงพีระมิด(ที่มองเห็นเหนือระนาบ S)



รูปที่ 2.11 An RGB cube showing a plane S of constant d

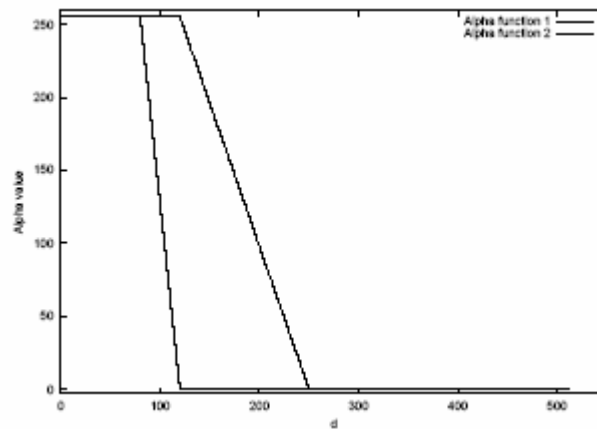
ที่กล่าวมาก่อนหน้านี้, สี(pixel) สามารถแบ่งแยกประเภทเหนือกว่าระนาบ S (และในพีระมิด)หรือต่ำกว่าระนาบ S ที่ซึ่งตำแหน่งที่แน่นอนของระนาบ S คือตั้งค่าโดยเลือกจากค่า $d_{\max}$ ที่เป็นตัวแปรเริ่มต้น สำหรับแต่ละ pixel ค่าสำหรับ d จะถูกคำนวณ, และ mask ที่สร้างจะสอดคล้องกับผลลัพธ์ของการเปรียบเทียบของ d ถึง $d_{\max}$

ในทางปฏิบัติ mask ที่เราสร้างนั้นจะไม่มี binary mask, แต่ค่าอัลฟาจะมีค่า 8-bit เมื่อเมื่อนำมารวมกับ foreground และ background (ใช้ mask) จุดภาพพิกเซลที่ส่งผลจะแก้ไขข้อความโดยการสอดคล้องเชิงเส้นของฉากหน้า และจุดภาพพิกเซลพื้นหลัง กับค่าอัลฟาจะทำการผสมตัวแปร ในปริบท, ค่าอัลฟากระทำเป็นค่าลักษณะโปร่งแสงสำหรับแต่ละจุดภาพพิกเซล

วิธีง่าย ๆ ของการวัดระยะ d โดยสร้างขอบเขตรอบๆตัวนักแสดง เพื่อพยายามในการแยกออกจาก background ตั้งแต่ที่เรามีค่า transparency 8 bit มันอาจจะทำให้ขอบรอบเรียบขึ้นโดยใช้ semi-transparent pixel บนขอบ ได้มีการทดลองอัลกอริทึมในการแก้ปัญหาในทางที่ดีขึ้น ระยะของ d คำนวณโดยใช้สมการที่ (2) ถูกใช้เป็น input เรียก alpha function ตัว alpha function จะ return ค่าระหว่าง 0 (transparent) และ 255 (opaque) โดยอาศัยวิธีจุด pixel จาก OP ใน RGB space สมการที่ (2) จะ return ค่าระหว่าง 0 (สำหรับสีเทาบริสุทธิ์) ถึง 510 (สำหรับสีน้ำเงินบริสุทธิ์) ซึ่ง R และ B มีค่าอยู่ระหว่าง 0 และ 255 ฟังก์ชันอัลฟาทั้งสองแสดงในรูปที่ 2.12

เหตุผลที่แสดงฟังก์ชันอัลฟาทั้งสองคือ อัลกอริทึมที่มีการปรับตัวแปรควบคุมรูปร่าง ของฟังก์ชันอัลฟา ซึ่งใช้อัลกอริทึมสำหรับรายละเอียดของห้องสีน้ำเงินในวิดีโอ ใน code นี้อัลฟาฟังก์ชันจะปฏิบัติกับ 512-entry ในทางปฏิบัติทางลาดเชิงเส้นที่แสดงในรูปที่ 8 มีการทำงานที่ดี

มาก การผลิตขอบรอบๆนักแสดงราบรื่น ซึ่งที่ตำแหน่งเริ่มต้นและตำแหน่งสิ้นสุดของทางลาดจะเลือกสีน้ำเงินออก



รูปที่ 2.12 Two sample alpha functions

ในรูปที่ 2.13 .ในแต่ละรอบ จะทำการเปรียบเทียบเพียงสองครั้ง ก็จะทราบว่าเป็นจุดที่ต้องการหรือไม่ ในการทำ lookup นั้นต้องการ สามโอเปอร์เรชั่นในการคำนวณหา index ที่ถูกต้องในตาราง ถ้าเป็นจุดที่ไม่ใช่สีน้ำเงินจะใช้เพียงสองโอเปอร์เรชั่น ในขณะที่จุดสีน้ำเงินจะต้องใช้ถึงห้า แล้วแต่ภาพ ผลลัพธ์จะอยู่ระหว่างสามและสี่โอเปอร์เรชั่นต่อพิกเซลโดยเฉลี่ย

```
// s points to the interleaved RGBA image
unsigned char *rp=s,*gp=s+1,
               *bp=s+2,*ap=s+3;
for (int i = 0; i < imgsize; i++)
{
    if (*bp > *rp && *bp > *gp )
        *ap = alpha_map[( *bp<1) - *rp - *gp];
    else *ap = 255;
    rp += 4; bp += 4;
    gp += 4; ap += 4;
}
```

รูปที่ 2.13 Two sample alpha functions

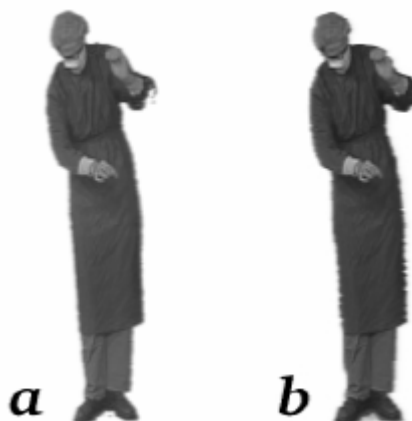
2.2.7 Image quality

มีหลายปัจจัยที่สามารถพิจารณาในการเปรียบเทียบคุณภาพของภาพที่ได้จากอัลกอริทึมของ software กับอุปกรณ์ hardware ในส่วนของ hardware นั้นสามารถเหลืงเอาไปได้ในขณะที่ถ้าเป็น software มันจะเอางาออก แต่ในการตั้งค่าในส่วนที่ 5 นั้นต้องการการย้ายออกของงา เพื่อการเปรียบเทียบที่ดีคือ diff- cult ในการ test ภาพจะต้องเอาผลลัพธ์มาเปรียบเทียบกันและให้เวลา 2 นาทีในการ fine-tuning

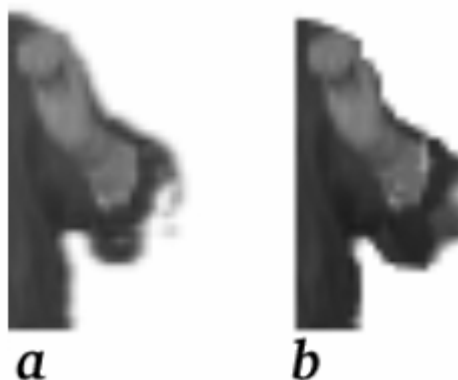


รูปที่ 2.14 Original Image

การสังเกตที่น่าสนใจขึ้น รูปที่ 2.15 โดยดูที่เหลืออยู่บนแขนของศัลยแพทย์(ที่ด้านมือขวาของรูปภาพ) ส่วนหนึ่งที่ hardware ได้key อย่างไม่ต้องโดยเอาบางส่วนของแขนออกไป (รูปที่ 2.15a)และ software keyer (รูปที่ 2.15b) สามารถจัดการส่วนนี้ได้ดีกว่าเล็กน้อย ได้ขยายให้ดูในภาพที่ 2.16 ซึ่ง software เองก็ผลิตรูปภาพได้ดีพอที่จะใช้กับโปรแกรมได้



รูปที่ 2.15 a) Hardware keyed, b) Software keyed



รูปที่ 2.16 Blow-up of elbow region: a) Hardware keyed, b) Software keyed

2.2.8 Performance

สำหรับส่วนนี้, วิธีคิด HSL เหมาะสมคือถูก bench markedรวมทั้งปรับปรุงอัลกอริทึม การทดสอบประมวลผลบน Intel Pentium II, ที่ความเร็ว 450MHz ในการทดสอบทั้งสองจะต้องใช้อัลกอริทึมในหน่วยความจำหลัก วิธีคิดใช้กับรูปภาพในหน่วยความจำหลัก, และเฟรมต่อที่สอง (FPS) การแบ่งแยกตามชั้นถูกคำนวณอาศัยเวลาที่มันหยิบให้ถูกยแจสิ่งหนึ่งพันรูปภาพนี้เวลาและ Frames Per Second (FPS)

Resolution	HSL Algorithm	Improve Algorithm
720 × 576	12.02 FPS	43.16 FPS
368 × 288	50.45 FPS	173.01 FPS
128 × 256	158.98 FPS	625.00 FPS

ตารางที่ 2.1 Software HSL vs Improved Algorithm, FPS

Resolution	HSL Algorithm	Improve Algorithm
720 × 576	4.98 Mpixels/s	17.90 Mpixels/s
368 × 288	5.15 Mpixels/s	18.34 Mpixels/s
128 × 256	5.21 Mpixels/s	20.48 Mpixels/s

ตารางที่ 2.2 Software HSL vs Improved Algorithm, Mp/s

ในตารางที่ 2.1 จะแสดงผลลัพธ์ของการรันอัลกอริทึมที่ภาพขนาดต่างกันขณะที่ตารางที่ 2.2 ก็แสดงเหมือนกัน แต่แสดงในอัตราหนึ่งล้านของ pixel ต่อวินาที (Mpixels) การปรับปรุงอัลกอริทึมที่แสดงนั้นมีความสามารถเพิ่มขึ้น 340% และสูงถึง 390% ในการปฏิบัติเมื่อเปรียบเทียบกับวิธีคิด HSL

2.3 VideoOCX

VideoOCX คือ ActiveX control ที่ทำให้นักเขียนโปรแกรมสามารถทำ video capture ได้ง่ายขึ้นและทำการประมวลผลรูปภาพเข้าไปในโปรแกรมได้ ตัว control ที่เข้ากันได้ส่วนมากเป็นอุปกรณ์ Video-for-Windows (VFW) เช่นเดียวกับกล้อง USB (Webcams) และ framgrabbers ที่ทำงานร่วมกับ CCD VideoOCX จะยอมให้จับภาพวิดีโอในหนึ่งขั้นตอนที่ง่ายดาย โดยต้องนำ VideoOCX เข้าไปวางในโปรแกรม เปลี่ยนคุณสมบัติและเพิ่มคำสั่งเล็กน้อย ก็สามารถใช้งานได้

2.3.1 ความต้องการพื้นฐานสำหรับใช้งาน VideoOCX

- Windows 95/98/ME/NT/2000/XP (TM)
- Development environments that supports ActiveX (TM) technology (e.g. Visual Basic (TM), Visual C++ (TM), Borland C++ (TM), Delphi (TM)...)
 - Video for Windows compatible image source (USB camera, framegrabber)
 - Frame rate and processing speed depend on your system performance

2.3.2 ความสามารถของ VideoOCX

- ฟังก์ชันที่ใช้งานง่าย code เพียงแค่ 3 บรรทัดก็สามารถใช้สำหรับเครื่องควิดีโอได้
- Capture color video frames เข้าไปหน่วยความจำด้วยความเร็วสูง (เช่น 25fps ที่ 768x576)
- เก็บ frame capture เป็น AVI sequence ในรูปแบบอื่นๆได้
- บันทึกภาพที่ capture ลงใน disk
- มี option แปลงภาพสีเป็นภาพขาวดำ
- เข้าถึงข้อมูลของ pixel RGB ได้
- รองรับอุปกรณ์วิดีโอ capture PAL/NTSC VIEW ได้มากกว่า
- รองรับกล้อง USB
- รองรับ RGB AVI sequence
- แปลงภาพต้นฉบับ (AVI, Video) โดยการปรับ control เพียงตัวเดียว

2.3.3 Method ต่างๆสำหรับการใช้งาน VideoOCX

- **Init Method**

Syntax : BOOL object.Init()

เป็นขั้นตอนแรกที่ VideoOCX จะเชื่อมต่อไปยังแหล่งข้อมูลภาพที่ระบุไว้ จะถูกเรียกเพียงครั้งเดียวจนกระทั่งมีการเปลี่ยนโหมดของแหล่งข้อมูลภาพ ในโหมด AVI (Mode = 1) จะแสดง DialogBox สำหรับเลือกไฟล์ขึ้นมาให้เลือกไฟล์วิดีโอฟอร์แมต AVI ที่จะใช้เป็นแหล่งข้อมูลภาพ ส่วนในโหมด VFW (Mode = 0) จะเป็นการเชื่อมต่อไปยังแหล่งข้อมูลภาพจากกล้อง เมธอดนี้จะส่งค่ากลับเพื่อบอกว่าทำการเชื่อมต่อไปยังแหล่งข้อมูลภาพสำเร็จหรือไม่

- **InitFromFile Method**

Syntax : BOOL object.InitFromFile(LPCTSTR filename)

เป็นขั้นตอนแรกที่ VideoOCX จะเชื่อมต่อไปยังแหล่งข้อมูลภาพใน AVI โหมด ได้แก่ ไฟล์วิดีโอฟอร์แมต AVI โคนจะระบุ Path ของไฟล์ผ่าน ตัวแปร filename เมธอดนี้จะส่งค่ากลับเพื่อบอกว่าทำการเชื่อมต่อไปยังแหล่งข้อมูลภาพสำเร็จหรือไม่

- **Close Method**

Syntax : BOOL object.Close()

เป็นขั้นตอนสุดท้ายเมื่อเลิกใช้แหล่งข้อมูลภาพ

เมธอดนี้จะส่งค่ากลับเพื่อบอกว่าทำการ Close สำเร็จมัย

- **Start Method**

Syntax : BOOL object.Start()

เป็นขั้นตอนแรกก่อนทำการดึงภาพจากแหล่งข้อมูลภาพ

เมธอดนี้จะส่งค่ากลับเพื่อบอกว่าทำการ Start สำเร็จมัย

- **Stop Method**

Syntax : BOOL object.Stop()

เป็นขั้นตอนหลังเลิกทำการดึงภาพจากแหล่งข้อมูลภาพ

เมธอดนี้จะส่งค่ากลับเพื่อบอกว่าทำการ Stop สำเร็จมัย

- **GetColorImageHandle Method**

Syntax : long object.GetColorImageHandle()

ใช้สำหรับจองหน่วยความจำสำหรับเก็บภาพเพื่อนำไปใช้ในการประมวลผลต่างๆ

เมธอดนี้จะส่งกลับค่าที่ใช้อ้างอิงถึงหน่วยความจำส่วนนี้ เพื่อใช้ส่งต่อไปให้ Method อื่นๆ ใช้ในการประมวลผลต่อไป

- **ReleaseImageHandle Method**

Syntax : void object .ReleaseImageHandle(long imagehandle)

ใช้สำหรับคืนหน่วยความจำที่เก็บภาพซึ่งอ้างอิงโดยตัวแปร imagehandle

- **CtlCapture Method**

Syntax : long object .CtlCapture(long imagehandle)

ใช้สำหรับจับภาพที่เฟรมปัจจุบันมาไว้ในหน่วยความจำที่อ้างอิงโดยตัวแปร imagehandle และเตรียมพร้อมสำหรับเฟรมถัดไป

เมธอดนี้จะส่งกลับค่าเป็นตำแหน่ง address เริ่มต้นของข้อมูลภาพในหน่วยความจำ ถ้าทำการจับภาพไม่สำเร็จจะส่งกลับค่าเป็น 0

- **CtlShow Method**

Syntax : void object .CtlShow(long imagehandle)

ใช้เมื่อต้องการนำภาพในหน่วยความจำที่อ้างอิงโดยตัวแปร imagehandle ขึ้นมาแสดงบน VideoOCX Control

- **SetPreview Method**

Syntax : BOOL object .SetPreview(BOOL show)

กำหนดการแสดงผลภาพตัวอย่างใน ViderOCX Control

- **GetImageWidth Method**

Syntax : long object .GetImageWidth(long imagehandle)

ใช้ดึงค่าความกว้างของภาพในหน่วยความจำที่อ้างอิงโดยตัวแปร imagehandle

- **GetImageHeight Method**

Syntax : long object .GetImageHeight(long imagehandle)

ใช้ดึงค่าความกว้างของภาพในหน่วยความจำที่อ้างอิงโดยตัวแปร imagehandle

- **GetImageDataSize Method**

Syntax : long object .GetImageDataSize(long imagehandle)

ใช้ดึงขนาดของหน่วยความจำของภาพในหน่วยความจำที่อ้างอิงโดยตัวแปร imagehandle

- **GetDataPointer Method**

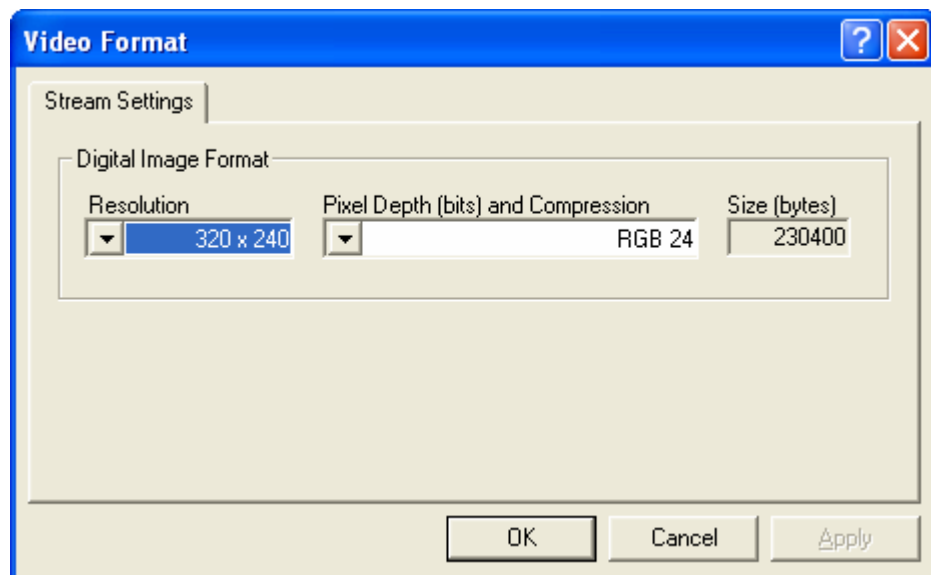
Syntax : long object .GetDataPointer(long imagehandle)

ใช้ดึงตำแหน่ง address เริ่มต้นของข้อมูลภาพในหน่วยความจำของภาพในหน่วยความจำที่อ้างอิงโดยตัวแปร imagehandle

- **ShowFormatDlg Method**

Syntax : BOOL object.ShowFormatDlg()

ใช้เมื่อต้องการแสดง DialogBox เพื่อปรับค่า ขนาดของภาพใช้ได้กับโหมด VFW เท่านั้น และจะต้องทำการ เรียกใช้เมธอด Init ก่อนและไม่อยู่ในช่วงที่กำลังทำการจับภาพ เมธอดนี้จะส่งกลับค่า เป็น TRUE ถ้าสามารถทำการแสดง Format Dialog ได้สำเร็จ

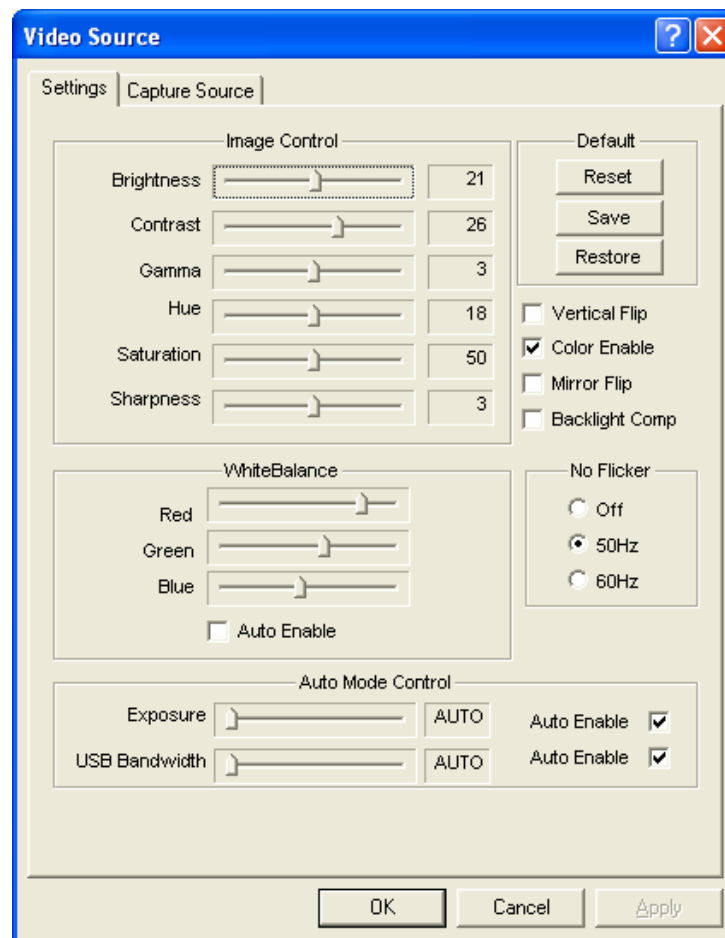


รูปที่ 2.17 แสดงหน้าต่าง Video Format

- **ShowSourceDlg Method**

Syntax : BOOL object.ShowSourceDlg()

ใช้เมื่อต้องการแสดง DialogBox เพื่อปรับค่าต่างๆเกี่ยวกับรูปภาพ ได้แก่ Brightness , Contast , Gamma , Hue , Saturation , Sharpness และค่าอื่นๆ ตามความสามารถของกล้อง เมธอดนี้จะส่งกลับค่า เป็น TRUE ถ้าสามารถทำการแสดง Format Dialog ได้สำเร็จ



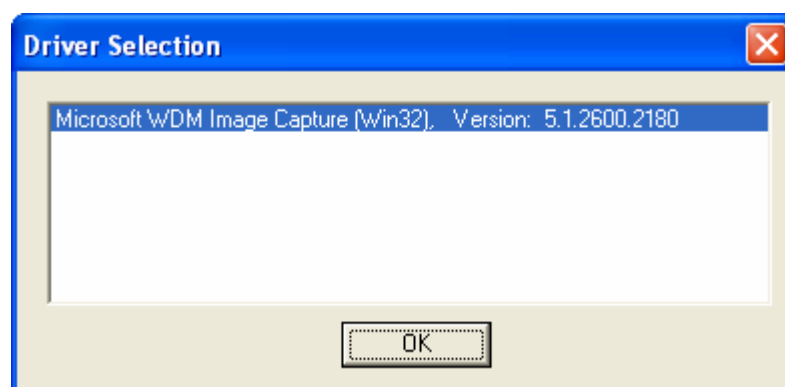
รูปที่ 2.18 แสดงหน้าต่าง Video Source

- **ShowDriverDlg Method**

Syntax : BOOL object.ShowDriverDlg()

ใช้เมื่อต้องการแสดง DialogBox สำหรับเลือก Driver

เมธอดนี้จะส่งกลับค่า เป็น TRUE ถ้าสามารถทำการแสดง Format Dialog ได้สำเร็จ



รูปที่ 2.19 แสดงหน้าต่าง Driver Selection

- **GetLastErrorString Method**

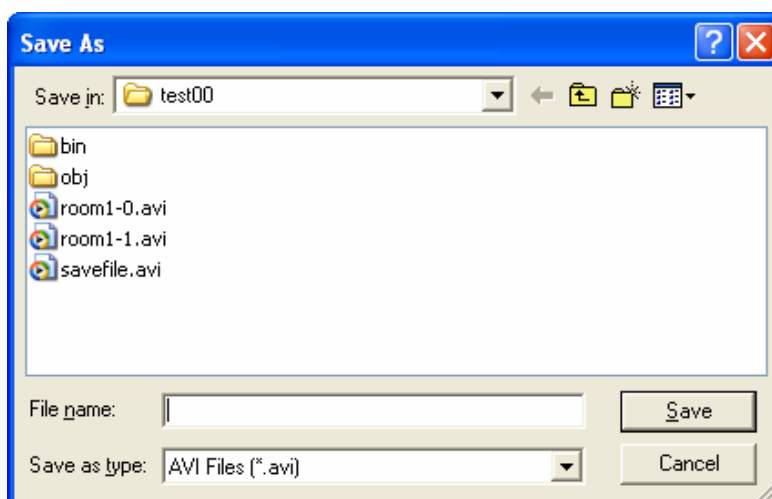
Syntax : LPCTSTR object.GetLastErrorString()

ใช้ดึงข้อความแสดงถึงความผิดพลาดล่าสุด ของ VideoOCX

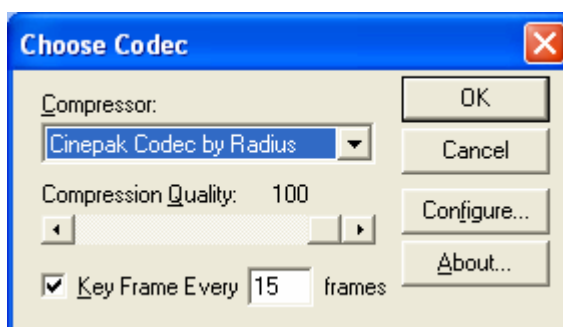
- **AVISaveFrameInit Method**

BOOL object.AVISaveFrameInit(String filename,
long imagehandle,
BOOL showcompressiondialog
)

ใช้เพื่อเปิดไฟล์ AVI หากกำหนด filename เป็น "" จะทำให้ VideoOCX แสดง DialogBox สำหรับ ตั้งชื่อไฟล์ที่ต้องการบันทึก ขนาดของภาพ จะถูกกำหนดตาม Imagehandle ที่ระบุไว้ หากกำหนด showcompressiondialog เป็น TRUE จะแสดง DialogBox สำหรับกำหนด ค่าในการทำ Compression



รูปที่ 2.20 แสดงหน้าต่างบันทึก



รูปที่ 2.21 แสดงหน้าต่าง Choose Code

- **AVISaveFrameClose Method**

Syntax : `BOOL object.AVISaveFrameClose()`

ใช้เมื่อต้องการสิ้นสุดการบันทึกไฟล์ AVI และปิดไฟล์

- **AVISaveFrameAdd Method**

Syntax : `BOOL object.AVISaveFrameAdd(long imagehandle)`

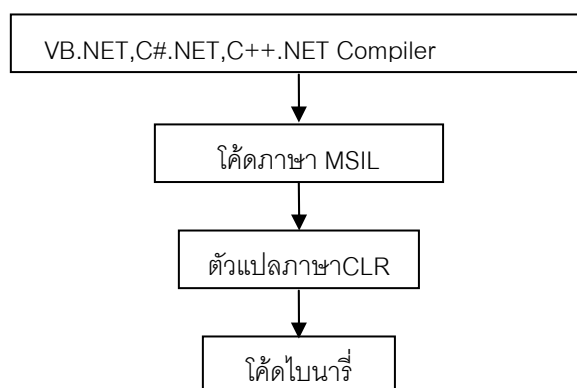
ใช้เพิ่มเฟรมภาพจากหน่วยความจำที่อ้างอิงโดยตัวแปร imagehandle ลงในไฟล์ AVI ที่ได้เปิดเตรียมไว้

2.4 Visual C#.NET

Visual studio.NET ถือเป็นเครื่องมือพัฒนา Application ตัวแรกที่จะออกมารองรับการทำงานตามแนวความคิดของสถาปัตยกรรม .NET Framework ประกอบไปด้วยเครื่องมือต่างๆดังนี้ Visual Basic.NET, Visual C#.NET, Visual C++.NET, Visual ASP.NET

ภาษา Visual C#.NET ถือเป็นภาษาที่เกิดขึ้นมาพร้อมกับแนวความคิดของการเขียนโปรแกรมในยุค .NET อาจกล่าวได้ว่าภาษา Visual C# คือภาษาต้นแบบในการเขียนโปรแกรม .NET นั้นเอง โดยที่ภาษาอื่นๆที่เกิดขึ้นมาก่อนหน้านี้จะต้องปรับตัวเข้าหา .NET ทั้งหมดซึ่งสามารถสังเกตได้ว่า ไวยากรณ์การใช้งานของแต่ละภาษานั้นล้วนแล้วแต่ ถูกปรับเปลี่ยนไปจากเวอร์ชันก่อนหน้านี้อย่างสิ้นเชิง

หลักการทำงานของตัวแปลภาษา C# ความสำคัญอีกอย่างหนึ่งของภาษาต่างๆใน .NET ก็คือตัวแปลภาษาหรือที่เราเรียกว่า compiler จากอดีตที่ผ่านมาเราจะพบว่าแต่ละภาษาจะมีตัวแปลภาษาเป็นของตัวเอง แต่สำหรับภาษาต่างๆที่อยู่ใน Visual studio.NET แล้ว Microsoft ได้ปรับปรุงตัวแปลภาษา เปลี่ยนไปอย่างสิ้นเชิง โดยไม่ว่าจะพัฒนา application ด้วยภาษาใดก็ตามจะอาศัยตัวแปลที่เรียกว่า CLR (Common Language Runtime) ทำหน้าที่แปลง code ไปสู่ภาษาเครื่อง



รูปที่ 2.22 แสดงหลักการทำงานของตัวแปลภาษา Visual C#

จากรูปที่ 2.22 เมื่อเกิดการแปลโค้ดที่มาจากภาษาใดๆก็ตามใน .NET จะอาศัย CLR ทำหน้าที่แปลออกมาเป็นภาษากลางที่เรียกว่า IL (Intermediate Language) ก่อน เมื่อได้โค้ดของ IL มาแล้ว ถ้าต้องการแปลออกมาเป็นภาษาเครื่องก็จะอาศัยหลักการทำงานของ Virtual Machine แปลภาษา IL อีกครั้งหนึ่ง โดยอาศัย Compiler JIT (Just-In-Time) ซึ่งมีข้อดีคือ สามารถใช้ภาษาอะไรก็ได้ที่นักเขียนโปรแกรมกับ .NET ก็ได้ผลลัพธ์เหมือนกัน หรืออาจจะสร้าง Application จากภาษาต่างๆ มากกว่าหนึ่งภาษาก็ได้ ทำให้ยืดหยุ่นต่อการพัฒนา Application แบบเป็นทีม